

工具链与工程

host/target/sysroot → 交叉编译 → ELF 分析 → Makefile → 工程目录规范

2.1 交叉编译全流程：从源码到板端可执行文件

学习目标

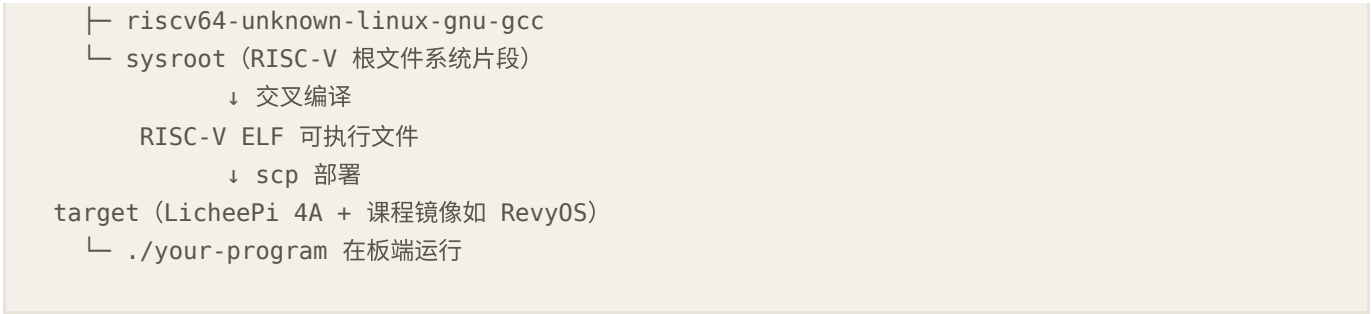
- 区分 host（编译主机）、target（运行设备）和 sysroot（目标根文件系统）
- 说明交叉编译与本地编译的差异，以及课程为何在主机上编译、在板端运行
- 查询交叉编译器目标三元组与 sysroot 路径，填写三要素对照表
- 使用交叉编译器构建最小 C 程序，确认产物为 RISC-V ELF，SCP 部署到开发板并运行
- 使用 `readelf` 查看 ELF 头、程序头中的关键字段
- 使用 `ldd` 查看动态可执行文件的共享库依赖

host、target、sysroot 与交叉编译

嵌入式 Linux 开发很少在开发板上直接编译大型程序。更常见的流程是：在性能更好的 PC（host）上生成二进制，再部署到开发板（target）执行。当 host 与 target 的 CPU 架构不同时，称为交叉编译（cross compilation）。

工具链除了编译器，还需要目标系统的头文件和库。这些文件按目标系统的目录结构组织，合称为 sysroot。编译时编译器在 sysroot 中查找头文件和库；链接时按目标 ABI 链接其中的库。

host (x86_64 Linux PC)
└─ 编辑器、make、rui



环境准备

项目	要求	检查方式
host 架构	记录主机 CPU 架构	<code>uname -m</code>
target 架构	板端为 RISC-V 64 位	<code>ssh user@board-ip uname -m</code>
交叉编译器	前缀 <code>riscv64-unknown-linux-gnu-</code>	<code>command -v riscv64-unknown-linux-gnu-gcc</code>
工程目录	<code>chapters/ch02/code/hello/</code>	<code>ls main.c Makefile</code>

关键区分： `riscv64-unknown-elf-gcc` 面向无操作系统环境（裸机）。本课程用户态程序应使用带 `linux-gnu` 的工具链。

操作步骤

步骤 1：确认 host 与 target

```
# 主机
$ uname -m; hostname

# 板端
$ ssh user@board-ip 'uname -m; hostname; head -n 5 /etc/os-release'
```

主机常见结果为 `x86_64`，板端为 `riscv64`。两者不同，说明需要交叉编译。

步骤 2：查看交叉编译器目标三元组与 sysroot

```
$ riscv64-unknown-linux-gnu-gcc -dumpmachine
$ riscv64-unknown-linux-gnu-gcc --version | head -n 1
$ riscv64-unknown-linux-gnu-gcc -print-sysroot
```

若 `-print-sysroot` 为空，工具链可能使用内置默认路径——用 `gcc -v` 输出查找，不要因此改用主机 gcc。

步骤 3：对比 host 与 target 运行环境

```
$ file /bin/bash
$ ldd /bin/bash | head -n 3
```

分别在主机和板端执行。观察动态链接器路径、库目录与架构差异。

步骤 4：交叉编译第一个程序

```
$ cd chapters/ch02/code/hello
$ make clean && make
$ file hello      # 应显示 RISC-V ELF
```

若 `file hello` 显示 `x86-64`，说明调用了错误的编译器。检查 `CROSS_COMPILE` 变量。

步骤 5：部署到板端并运行

```
$ scp hello user@board-ip:~/
$ ssh user@board-ip 'chmod +x ~/hello && file ~/hello'
$ ssh user@board-ip './hello'
```

预期输出： `Hello, RISC-V Linux!`

为什么不在主机验证：x86_64 主机无法运行 RISC-V 二进制。正确验收必须在板端执行。

步骤 6：ELF 分析

```
$ readelf -h hello      # Machine = RISC-V?
$ readelf -l hello      # LOAD 段布局
$ ldd hello             # 动态库依赖（静态链接则注明）
```

字段	含义	期望
Class	ELF 位数	ELF64
Machine	CPU 架构	RISC-V
Type	文件类型	EXEC 或 DYN

可选：在板端执行 `ldd ~/hello` 与 host 结果对比。

课堂练习

- 判断对错并改正：「在 x86_64 笔记本上 `gcc hello.c` 得到的程序，可以直接 scp 到 LicheePi 4A 运行。」
- 若 `scp` 成功但板端提示 `cannot execute binary file`，依次检查哪些项目？
- `readelf -h` 中 `Machine` 为 `X86-64` 时，构建过程哪里出了问题？

本节成果

- host / target / sysroot 对照表
- 交叉编译器目标三元组与 sysroot 路径记录
- 板端可运行的 Hello World
- 可复用的「编译 → scp → 运行」命令链
- `readelf -h` / `readelf -l` 关键字段摘录
- `ldd` 或静态链接说明

2.2 Makefile 与工程目录规范

学习目标

- 编写包含目标、依赖、命令的基本 Makefile 规则，理解 Tab 缩进
- 使用变量定义 `CC`、`CFLAGS`、`CROSS_COMPILE`，避免硬编码
- 实现 `make`、`make clean` 及 `.PHONY` 伪目标
- 将源码、头文件与构建产物分别放入 `src/`、`include/`、`build/`
- 使用多文件工程 Makefile 完成编译、清理与板端部署

Makefile 基础

Makefile 描述「什么文件由什么文件生成、执行什么命令」。`make` 根据文件时间戳决定是否重新编译。

```
Makefile
CROSS_COMPILE, CC, CFLAGS
↓
hello: main.c
    $(CC) $(CFLAGS) -o $@ $^
↓
make / make clean
```

统一工程目录

单文件 `hello` 适合入门，真实项目需拆分多个 `.c` 与公共头文件：

```
project-template/
├─ Makefile
├─ include/      ← 对外头文件
├─ src/          ← 源文件
└─ build/        ← 产物 (.o、可执行文件)，可删可重建
```

课程模板位于 `chapters/ch02/code/project-template/`。`make` 后生成 `build/app`；`make clean` 删除整个 `build/`。

```
include/greet.h
src/main.c └─
src/greet.c ┤─> build/src/*.o ─> build/app
Makefile ──┘
```

操作步骤

阶段 A：单文件 Makefile

```
$ cd chapters/ch02/code/hello
$ cat -A Makefile      # 可见行尾与 Tab (^I)

$ make clean && make
$ make                  # 第二次应提示已是最新
$ touch main.c
$ make                  # 应重新编译

$ make clean && ls hello 2>&1 # hello 应不存在
$ make CFLAGS='-Wall -Wextra -O0 -g' # 命令行覆盖变量
```

阶段 B：多文件工程

```
$ cd chapters/ch02/code/project-template
$ find . -type f | sort

$ make clean && make
$ file build/app      # RISC-V ELF

$ scp build/app user@board-ip:~/
$ ssh user@board-ip 'chmod +x ~/app && ./app'
# 预期输出: Hello, RISC-V Linux!

$ make clean && test ! -d build && echo "build removed"
```

工程习惯： build/ 可安全删除且能一键重建——这为后续实验的版本管理打下基础。将 build/ 加入 .gitignore ， src/ 和 include/ 纳入版本库。

课堂练习

1. 解释以下错误为何导致 `make` 失败：命令行前用空格而非 Tab；目标未声明依赖且产物已存在；`clean` 未标记 `.PHONY` 且目录下有同名文件。
2. 为何 `build/` 加入 `.gitignore`，而 `src/` 和 `include/` 纳入版本库？

本节成果

- 能解释 `hello` Makefile 各变量与规则含义
- `make` / `make clean` / 增量构建演示记录
- 符合 `src/include/build` 约定的可运行工程
- 板端运行 `app` 的记录
- 可复用的 Makefile 模板路径：`chapters/ch02/code/project-template/`

2.3 选读：CoreMark 算力基准

选读

学习目标

- 了解 CoreMark 嵌入式基准测试的用途与局限
- 在板端获取、构建并运行 CoreMark，读懂分数含义
- 说明基准分数与真实应用性能（I/O、外设、网络）之间的区别

本讲为选读，不阻塞主线实验验收。课程综合项目验收不依赖 CoreMark 分数。价值在于建立「这块板子 CPU 大概什么量级」的直觉。

CoreMark 是什么

CoreMark 是 EEMBC 发布的轻量级 CPU 基准，测试列表操作、矩阵运算、状态机等核心逻辑，结果以 CoreMark 分数或 CoreMark/MHz 报告。它衡量 CPU 纯算力——不代表 GPIO 抖动、网络吞吐或存储速度。

操作步骤

```
# 获取源码（从 EEMBC 官方或课程镜像站）
# 选择 Linux/POSIX 移植，不是裸机移植

$ export CC=riscv64-unknown-linux-gnu-gcc
$ make PORT_DIR=linux XCFLAGS="-O2" link
$ file coremark.exe

$ scp coremark.exe user@board-ip:~/
$ ssh user@board-ip 'chmod +x ~/coremark.exe && ./coremark.exe'
# 记录 Iterations、Total time、CoreMark 分数
```

理解分数

编译器版本、`-O` 级别、glibc/musl、是否固定 CPU 频率都会导致分数差异。只作同环境前后对比，不追求绝对名次。CoreMark 高分并不意味着 MQTT 或传感器实验一

定更流畅——I/O 和网络延迟不在基准测试范围内。

课堂练习

列举三项不会被 CoreMark 单独反映、但在本课程实验中很重要的系统能力。

本节成果

- 可选：CoreMark 板端运行日志
- 可选：关于「算力基准 vs 课程应用」的简短评述